

Cloud Programming and Operations 2025-26

Capstone Project: EuroTransit Marketplace

Building, delivering, observing, and hardening a resilient cloud-native system

Introduction

This is the final, integrative project of the course. Everything you practised across Lab01 to Lab05 (build/test/CI, containerization, Kubernetes workloads, ingress and TLS, operators, GitOps delivery, and observability) now converges on one system that you must **design for failure, deliver through Git, observe under load, and prove resilient in a live presentation**.

The labs taught you to *make a system run* and *make it understandable*. The capstone asks the harder question the course has been building toward:

Stability is not a property. It is an outcome. Can you produce that outcome on purpose, under partial failure, and prove it?

You will build **EuroTransit**, a small but realistic multi-service marketplace. The point is not feature richness. The point is operational behaviour: how the system holds together when one service slows down, when a Pod dies mid-request, when the database fails over, when a network partition isolates a component, and when you deploy a new version into all of this without taking the system down.

This project is also where **agentic coding is explicitly permitted and expected**, under the conditions described below. You will be graded far more on the judgments only your team can make than on the artifacts an agent can generate.

Learning outcomes

By the end of this project, your team should be able to:

- decompose a domain into cooperating services with explicit synchronous and asynchronous boundaries;
- justify a consistency model (CAP / PACELC) for a contended resource and implement it;
- implement an asynchronous processing pipeline using Kotlin coroutines / Flows, with correct cancellation and shutdown semantics;
- make cross-service operations safe under retries, duplicates, and partial failure (idempotency, circuit breakers, bulkheads, backpressure, graceful degradation);

- define meaningful SLOs and the SLIs, dashboards, and symptom-based alerts that protect them;
- deliver changes through GitOps with at least two progressive-delivery strategies (canary and blue/green);
- design and run controlled chaos experiments and demonstrate that partial failures stay contained;
- reason about the security and governance posture of **non-human actors** (CI, controllers, and coding agents) inside your delivery loop;
- run a live incident during presentation, recover, and produce a blameless postmortem.

The EuroTransit domain

EuroTransit is a marketplace for selling train tickets (you may reskin the domain freely; what matters are the interaction patterns). It is composed of **five services**:

Service	Responsibility	Interaction style
Catalog	Lists products/offers; mostly reads	Synchronous API; tolerant of staleness
Orders	Orchestrates the purchase workflow	Synchronous entry plus asynchronous pipeline
Inventory	Tracks finite seats; the contended resource	Synchronous reservation plus async events
Payments	Authorizes payment; must not double-charge	Synchronous call with strict idempotency
Notifications	Sends confirmations	Fully asynchronous; failure must degrade gracefully

Required interaction shape:

- A public **API gateway / ingress** (Traefik, from Lab03) is the single north-south endpoint.
- **Synchronous APIs** between client → gateway → Catalog/Orders, and Orders → Inventory/Payments where a decision must be made *now*.
- **Asynchronous communication** via **Kafka (deployed with the Strimzi operator)** for the order pipeline and all eventing (e.g. `order-placed`, `inventory-reserved`, `payment-authorized`, `order-confirmed`, `notification-requested`).
- **Orders** owns a PostgreSQL database managed by the **CloudNativePG operator** (from Lab03).
- **Notifications** must be able to fail entirely without failing checkout (graceful degradation).

The "money path" (placing an order, reserving inventory, authorizing payment, confirming) is the critical path. Your SLOs, dashboards, and chaos experiments must center on it.

What you must demonstrate (the four pillars)

The project is organized around four pillars. Each maps to course modules and to specific Definition-of-Done items.

Pillar A: Distributed design and asynchronous execution

- A clear service decomposition with documented synchronous vs. asynchronous boundaries and a justification for each.
- The **order pipeline implemented asynchronously** using **Kotlin coroutines / Flows**: an order placed via a synchronous API returns quickly, while reservation, payment, and confirmation proceed through Kafka-driven stages.
- Correct async lifecycle behaviour, demonstrated rather than claimed:
 - structured concurrency (a **CoroutineScope** as a failure domain);
 - cooperative cancellation on **SIGTERM** / Pod shutdown (in-flight work finishes or is cleanly cancelled, with no orphaned tasks and no double-processing);
 - readiness flips to "refusing traffic" while in-flight work drains.
- A short written analysis: where does async **reduce cost / scaling pressure** in EuroTransit, and where would it **not** help (CPU-bound work)? Reference the blocking-vs-suspending model from the async lecture.

Pillar B: Consistency under contention (the inventory problem)

Inventory is the one place where correctness genuinely conflicts with availability. Two customers must not buy the last seat.

- Choose and **justify a consistency model** for inventory reservation using **CAP / PACELC** terms. State explicitly what you sacrifice in a partition and what latency/consistency trade-off you accept when there is no partition.
- Implement it (e.g. conditional/atomic reservation in PostgreSQL, a reservation state machine, or a documented reconciliation strategy with compensating actions).
- **Idempotency is mandatory** across the money path: a duplicated **order-placed** event or a retried payment authorization must not double-reserve or double-charge. Use idempotency keys / deduplication and document the scheme.
- Demonstrate, under a chaos experiment, that the invariant ("never oversell") holds even when messages are duplicated or a Pod dies mid-reservation.

Pillar C: Resilience engineering

The system must **contain** partial failure rather than propagate it.

- **Circuit breakers** on cross-service synchronous calls (e.g. Orders → Payments), with a defined open/half-open policy and a safe fallback (cached, degraded, queued, or an explicit error rather than an unbounded hang).

- **Bulkheads**: isolated resource pools so a slow downstream cannot exhaust shared threads/connections and take down unrelated flows.
- **Timeouts and bounded retries with backoff plus jitter** on every remote call. No unbounded waits; no naive retry storms.
- **Backpressure / load shedding**: under overload, the system refuses work in a controlled way (e.g. HTTP 429) rather than queuing to collapse.
- **Graceful degradation**: for example, Notifications can be fully down and checkout still succeeds.
- Kubernetes-level resilience configured deliberately rather than defaulted:
 - meaningful **startup / readiness / liveness** probes (liveness must not fail on transient downstream issues);
 - **PodDisruptionBudgets**;

Pillar D: Delivery, observability, and proof under failure

- **GitOps delivery** continues from Lab04: CI builds/tests/publishes immutable images and updates the configuration repo; **Argo CD reconciles**. CI must not hold cluster credentials.
- **Two progressive-delivery strategies**, both demonstrated:
 - **Canary** (route a small fraction of traffic to a new version, watch SLIs, promote or abort; rely on TraefikService for implementation);
 - **Blue/green** (stand up the new version, switch traffic, keep fast rollback).
 - Discuss where the remaining two DORA-aligned strategies (all-at-once, rolling) would fit and why you did not use them on the critical path.
- **Observability** continues from Lab05: per-service RED dashboards, infrastructure USE/Golden-Signals, and **symptom-based alerts** tied to your SLOs (alert on burn / user-visible symptoms, not CPU).
- **SLOs** for the critical path: at minimum a latency SLO and a success-rate SLO for checkout, with the SLIs that measure them and an error-budget statement.
- **Distributed tracing** across the money path (request → gateway → Orders → Inventory/Payments → Kafka stages), so you can answer "where did this order spend its time, and where did it fail?"

Chaos engineering

Resilience that has never been tested is a hypothesis, not a property. You will validate yours.

Controlled chaos experiments (during development)

Using **Chaos Mesh** (or an equivalent), design and run experiments as *hypotheses*, following the scientific method from the resilience lecture: define steady state (your SLIs), state a hypothesis, inject one failure, observe, conclude. At minimum:

1. **Latency injection** into Payments: does the Orders circuit breaker open and the fallback engage, while Catalog browsing stays healthy?
2. **Pod kill** on Inventory mid-reservation: does idempotency plus the reservation model prevent oversell or double-charge?

3. **Node / AZ-style disruption:** do PDBs and topology spread keep the critical path available?
4. **Network partition / Kafka disruption:** does the async pipeline recover and converge once the partition heals? Does anything get lost or duplicated?
5. **Pod-failover of the CloudNativePG primary:** what is the observed impact on checkout, and does the system recover within your stated RTO?

Each experiment produces: the hypothesis, the steady-state definition, what you observed in your own dashboards, whether the hypothesis held, and what you changed if it didn't.

Agentic coding policy

Agentic coding tools are **permitted and encouraged** for this project, under three rules that mirror how non-human actors are treated everywhere else in the course.

1. **You may use agents to produce artifacts** (service scaffolding, Helm templates, dashboards, manifests, test harnesses). You may **not** outsource the judgments: the service decomposition, the consistency model, the SLO definitions, the failure-mode mapping, the chaos hypotheses, and the postmortem must be authored and owned by the team.
2. **The agent is an actor in your system, and you must reason about its blast radius.** If a coding agent can open pull requests against the configuration repository that Argo CD reconciles, it is inside your control loop. Document: what credentials/permissions it has (least privilege), whether its changes require human or policy-as-code review before merge, and what the worst case is if it proposes a bad change. A short threat-model paragraph is required.
3. **You must verify, and you must show where the agent was wrong.** Maintain a brief `docs/agent-log.md` recording at least **three concrete cases** where an agent-produced artifact was incorrect, unsafe, or subtly wrong (e.g. a liveness probe checking a downstream dependency, a cause-based alert, an over-permissive ServiceAccount, a non-idempotent handler) and how your team caught and corrected it. **This is graded.** Finding the agent's mistakes is the core skill the capstone certifies.

You will not be penalized for using agents heavily. You will be penalized if no one on the team can explain, defend, and operate what was built.

Repository and delivery model

Continue the two-repository model from Lab03/Lab04:

- **Application repository:** source for the five services, tests, CI workflows, `justfile`.
- **Configuration repository:** Helm charts/manifests for the application and the Argo CD `Application(s)`; platform bootstrap; SLO/alert definitions; team docs.

Platform components installed once per cluster (Traefik, cert-manager, CloudNativePG, Strimzi, Sealed Secrets, Argo CD, kube-prometheus-stack, Chaos Mesh) are shared infrastructure, not per-app workloads. Secrets live in Git only as **SealedSecrets** (Lab04). CI updates desired state; **Argo CD deploys**.

Definition of Done

Your team **authors and versions** its own DoD in `docs/capstone-dod.md`, but it must cover at least the following. Keep each item short and operational.

- Design & async (Pillar A)
- Consistency (Pillar B)
- Resilience (Pillar C)
- Delivery, observability, proof (Pillar D)
- Chaos experiments
- Agentic coding

Suggested team roles (5 people, rotate, do not silo)

Roles coordinate ownership; **everyone must understand the full money path end to end**.

- **Domain & async owner**: service boundaries, the coroutine/Flow order pipeline, shutdown semantics.
- **Data & consistency owner**: CloudNativePG, inventory consistency model, idempotency.
- **Resilience owner**: circuit breakers, bulkheads, probes, PDBs, HPA, Chaos Mesh experiments.
- **Delivery owner**: GitOps, Argo CD, canary/blue-green, Strimzi/Kafka wiring.
- **Observability & verification owner**: SLOs, dashboards, alerts, tracing, and custodian of the agent-verification log.

As in every lab: roles are for coordination, not for building knowledge silos.

Deliverables

1. The two repositories (application plus configuration), with history that shows GitOps-driven delivery.
2. `docs/` containing: the DoD, the design/consistency justification, the SLO definitions, the chaos-experiment reports, the postmortem, the agent threat-model paragraph, and `agent-log.md`.
3. A recorded **demo of about 5 min** (link committed in the repo) showing: the running system, the dashboards answering operational questions, a canary and a blue/green deployment, and at least one alert firing under an injected failure.

Once the system is ready, please contact us to schedule a **live presentation**. You will present your system and answer the technical questions related to the course material (**oral examination**).

Project evaluation

Grading weight follows the difficulty the agentic era exposes: **judgment over production**.

- **Resilience proven under failure** (chaos experiments).
- **Consistency & idempotency correctness** on the money path.
- **Observability that actually drives decisions** (not dashboard volume).
- **Delivery discipline** (GitOps, progressive delivery, no CI cluster creds).
- **Async correctness** (cancellation/shutdown), and the cost reasoning behind it.
- **Verification & ownership**: the agent-mistake log, the postmortem, and your ability to defend the system you delivered.

A system the agent largely wrote can still earn top marks **if your team understood it, instrumented it, broke it on purpose, and brought it back**.

Final note

This capstone deliberately closes the loop the course has drawn from its first slide: **declare intent, reconcile continuously, assume failure, observe everything, and own the outcome**. The tools will keep changing, agents included. The judgment will not. That judgment is what you are here to prove.